MINDFULLY YOURS · AI SERVICE

Performance & Scalability Plan

How the system stays fast and reliable as the number of users grows — the approach, the standard practices behind it, and an honest account of where we are today

> **Written for:** the client, to be read and explained by the engineering lead.
> **Question it answers:** “When the system grows and many users arrive, how will it stay fast (latency) and keep working (performance at scale)?”
>
> **Prepared:** 5 October 2026 · **Basis:** the project’s architecture and latency documents and the current backend code.
> **Status of figures:** latency numbers are measurements from our own tests. Capacity figures are illustrative until a formal load test is run, and are marked as such.

1. The short answer (read this page first)

IN ONE PARAGRAPH

The system is built so that **more users means adding more identical copies of the AI service, not making one copy bigger**. Each conversation's memory is kept in a shared fast store, so any copy can serve any user's next message. The slowest parts of a reply — the language model, speech-to-text and text-to-speech — are specialist vendor services that are designed to serve many customers at once. Our own logic (safety check, rule matching, knowledge lookups) takes milliseconds. The knowledge base is small and does not grow with the number of users. So the work ahead is standard, well-understood engineering: run several copies, protect the database, manage vendor limits, measure everything, and test under load before launch.

Five points to make to the client

1. **Speed and scale are two different questions.** Speed is how fast one person gets a reply. Scale is whether a thousand people at once all still get that same speed. We address them separately (Sections 2 and 5).
2. **The slow part is not our code.** In a reply, almost all the time is spent waiting on external AI services. Our own safety and rule logic is near-instant. That is good news for scaling — it means our part does not get slower as traffic grows.
3. **The knowledge base does not grow with users.** It is about 8,500 records whether we have 100 users or 100,000. What grows is traffic and conversation history, which are handled with standard techniques.
4. **Safety is never traded for speed.** Under heavy load we slow or switch off non-essential features first (avatar, then voice quality), never the safety check (Section 8).
5. **Honest status:** the design is scale-ready and the single-copy performance is good (about 1.7 s for a full text turn against a 1.5–2.5 s target). We have **not yet run a formal load test** and several hardening items are still to do. They are listed plainly in Section 10 with a plan and order of work.

The latency target we hold ourselves to

Mode	Target	Measured so far
Text reply	First words within ~1.5–2.5 s	~1.7 s on a tested full turn including retrieval and triage
Danger-phrase (safety) reply	As fast as possible	~0.6 s (no AI writing step is needed)
Voice: speech-to-text	Partial words streaming	Partial text in well under 1 s
Voice: text-to-speech first sound	As early as possible	~0.38–0.55 s to first audio
Voice: whole turn (speak → hear reply)	1.5–2.5 s	A real reply was received in a few seconds in live tests; per-stage timing for voice is not yet instrumented

All of the above are single-user measurements from our test environment. They show the speed of one conversation; they do not yet prove behaviour with many at once. That is what the load test in Section 9 is for.

2. Two different questions: latency and capacity

These are often mixed up. A simple analogy: a restaurant.

Latency = how long *my* meal takes

The time from a user finishing speaking or typing to the first part of the reply. Improved by a faster kitchen workflow: streaming, caching, doing steps in parallel, keeping connections open.

Capacity = how many diners at once

How many conversations can run simultaneously before replies slow down or fail. Improved by more tables and more cooks: more copies of the service, a database that can take the load, and enough supplier (vendor) capacity.

The risk at scale is that as capacity is stretched, latency quietly gets worse. A well-designed system keeps latency flat as users grow by adding capacity *before* it is stretched, and by measuring the slowest 5% of replies (not just the average).

Why the “slowest 5%” matters more than the average

If the average reply takes 1.7 s but 1 in 20 takes 6 s, a person in distress hits that slow reply regularly over a long conversation. So we track **P50** (typical), **P95** (slowest 1 in 20) and **P99** (slowest 1 in 100) for every stage, and set targets on the tail, not just the middle.

The standard approach to both

Goal	Standard practice
Lower latency for one user	Stream every stage; cache repeat work; run independent steps in parallel; keep connections to vendors open; host close to users and vendors; shrink what we send to the language model; mask unavoidable pauses with natural filler phrases.
More users at once	Stateless service copies behind a load balancer; shared fast session store; database connection pooling and read replicas; autoscaling on real load; managing vendor rate limits with queues, retries and fallbacks; load testing before launch.
Staying reliable under stress	Timeouts, retries with back-off, circuit breakers, graceful degradation, health checks, multi-zone deployment, alerting on latency and errors.

3. Where the time goes in one reply — and what that means at scale

A single reply passes through several stages. Some are our own code; some are external services. This split drives the whole scaling strategy.

Stage	Owner	Typical time	What happens as users grow
Speech-to-text (voice)	Vendor (Sarvam)	Streams; partials <1 s	Vendor scales; we manage concurrent-connection limits and quotas.
Turn the message into a meaning-fingerprint (embedding)	Vendor (OpenAI)	~0.3–0.6 s (network-bound)	One call per turn; the largest single contributor we control. Needs rate-limit management, short timeout, caching of repeat messages.
Cache lookup	Ours (Redis)	Milliseconds	Scales easily; managed Redis.
Silent understanding (signal matching)	Ours (Postgres)	Single to low-double-digit ms	Read-only queries on a small dataset — ideal for replicas and pooling.
Safety check	Ours (Postgres)	Milliseconds	Must stay fast and never be dropped. Protected first (Section 8).
Routing rules and tier score	Ours (code)	Microseconds–ms	Pure computation; scales with CPU. (An occasional small AI check for “how long / affecting life” adds ~0.25 s only on turns that still need it.)
Writing the reply	Vendor (Sarvam LLM)	Largest share; first words stream early	Vendor capacity and rate limits; our fallback provider and queueing matter most here.
Output check	Ours	Milliseconds	Scales with CPU.
Text-to-speech (voice)	Vendor (Sarvam)	~0.38–0.55 s to first audio	Vendor capacity; streaming sentence by sentence.
Audit log, session memory update	Ours	Off the critical path	Done in the background so it never delays a reply.



Illustrative proportions only, to show the shape: vendor calls dominate; our own logic is a small slice. Exact stage timings are recorded for every turn by the service and will replace this sketch with real P50/P95/P99 figures after load testing.

> **SAY THIS**
>
> “Most of the time in a reply is spent waiting on the AI vendors, and those are built to handle many customers. The parts we own — the safety check, the rules, the knowledge lookups — are already measured in milliseconds. So adding users mostly means adding service copies and managing vendor limits, not rewriting the engine.”

Why the knowledge base is not the scaling problem

The knowledge base holds about **8,500 records** (about 8,300 with a meaning-fingerprint). It is read-mostly, versioned, and changes only when the clinical team ships an update. Searching a

few thousand records takes milliseconds. Specialised vector databases only pull clearly ahead at millions to billions of records — we are about **three to five orders of magnitude below that**. User growth does not make the knowledge base bigger. What grows with users is the volume of *searches per second* and the volume of *conversation records saved*, which are handled by pooling, replicas and partitioning (Section 5).

4. The architecture that makes scaling possible

4.1 The layers

Client apps	Mobile and web. Connect to the backend; voice goes through LiveKit.
Product backend (NestJS)	Logins, bookings, journaling. Separate deployable — scales independently.
AI service (FastAPI)	Conversation engine, triage, safety interlock, provider adapters. **Stateless** — this is what we copy to scale.
Voice agents (LiveKit)	A worker pool that joins voice rooms and calls the same conversation engine. Scales separately from text.
Shared state	Redis (conversation memory and cache) and PostgreSQL with pgvector (knowledge base, sessions, audit). One data spine; no second vector database to keep in sync.
External AI vendors	Language model, speech-to-text, text-to-speech, embeddings — behind a swappable adapter layer.

4.2 Four design decisions that make scaling straightforward

Decision	Why it matters at scale
The AI service is stateless	A conversation's running state lives in Redis, not in the memory of one server. Any copy can serve any user's next message. Adding or removing copies, or losing one, does not lose conversations. This is the single most important property for horizontal scaling.
One conversation engine for text and voice	Voice and text call exactly the same function. We tune and scale one thing, and the safety behaviour is identical in both.
Vendors sit behind adapters	If a vendor is slow, limits us, or fails, we can route to a fallback without touching business logic. (Any substitution still needs the client's written consent under the agreement, and a fresh latency benchmark.)
Audit and summaries run in the background	Writing the audit trail never delays a reply, so a busy database does not make users wait.

4.3 Scaling picture

GROWTH PATH

Launch: 2+ copies of the AI service across two zones, one managed Postgres with a standby, managed Redis, a pool of voice-agent workers.
Growth: autoscaling copies on live load, Postgres read replicas for knowledge lookups, connection pooler in front of the database, voice workers autoscaled by active calls.
Large scale: partitioned audit tables, a semantic cache, optionally a self-hosted embedding model, and — only if measurements justify it — a dedicated vector engine.

5. Layer by layer: what we do and the standard practice behind it

5.1 The AI service (application tier)

Standard practice	How we apply it
Horizontal scaling behind a load balancer	Run multiple containers of the same service; the load balancer spreads requests. Containerised already (Docker).
Asynchronous, non-blocking code	The service is built on async Python (FastAPI). While one conversation waits on a vendor, the same process serves others — one copy can hold many simultaneous replies because the work is mostly waiting, not computing.
Multiple worker processes per container	Run several worker processes per container to use all CPU cores. (Today it runs a single process — a planned launch change.)
Autoscaling on real signals	Add copies automatically on concurrent requests and CPU, not just time of day; scale *out* early, scale *in* slowly so streaming replies are not cut off.
Health checks and graceful shutdown	Instances are removed from rotation before they stop, and given time to finish streaming replies in progress.
Multi-zone deployment	Copies spread over at least two availability zones in the Mumbai region so one zone's failure does not take the service down.

5.2 Conversation memory and caching (Redis)

- **What it holds:** each conversation's running state (turn count, accumulated signals, the 0–100 score) and the response cache.
- **Two-tier cache:** an *exact-match* cache (repeat questions answered instantly) and a *semantic* cache (near-duplicates). The exact cache is built and working. The semantic cache is designed but not yet implemented.
- **Safety rule:** a cache hit may skip the slow work (retrieval and writing), but **never** the safety check.
- **At scale:** use a managed, replicated Redis service in the same region; set expiry times so memory does not grow without limit; add clustering when a single node's memory or throughput is reached.
- **Caching in mental-health context:** a cache must be used carefully. Only generic, non-personal replies are suitable for caching, and cache keys are scoped by language. Anything personal to a user is never shared between users.

5.3 The database (PostgreSQL + pgvector)

Standard practice	How we apply it
Connection pooling	Many service copies each opening many connections can overwhelm a database. A pooler (such as PgBouncer or the cloud provider's equivalent) sits in between and shares a small number of real connections. We also set explicit pool sizes per copy.

Read replicas	Knowledge-base lookups (signals, red flags, rules) are read-only. They can be served from replicas, leaving the primary free for writes. Because the knowledge base is versioned and rarely changes, replica lag is not a clinical concern.
Right index for vector search	At today's ~8,500 records an exact search takes milliseconds and is perfectly accurate. As the knowledge base grows, we add an approximate-nearest-neighbour index (HNSW is the usual choice) and re-check that safety matches are not weakened by the approximation. See the honest note below.
Partition high-growth tables	The audit log grows with every turn from every user. Partition it by date so queries stay fast and old partitions can be archived or dropped under the retention policy.
Keep per-user data off the hot path	The reply path reads the small knowledge base and Redis. Heavy user history is written asynchronously and read only for reporting.
Backups, standby and failover	Managed Postgres with a synchronous standby in a second zone and point-in-time recovery. Required both for reliability and for data-protection obligations.

HONEST NOTE ON THE VECTOR INDEX

Earlier design notes mention an approximate vector index. In the current code, no dedicated vector index is defined — searches are exact scans. That is correct and fast for ~8,500 records, and exact results are actually a safety advantage today. It is on the hardening list to add and benchmark the right index *before* the knowledge base grows, and to test that safety-critical matches are unchanged by it.

pgvector or a dedicated vector database?

We use pgvector inside the existing database. It performs well to tens of millions of vectors; we are at ~8,500. It keeps the matching vector and the exact clinical script in **one query and one transaction** (no risk of two systems drifting apart), and it is one fewer system to secure for data-protection purposes. A dedicated engine such as Qdrant is a legitimate future option if the knowledge base grows by orders of magnitude; the decision is not a one-way door. Cloud-only vector services were excluded because they cannot meet data-residency requirements.

5.4 AI vendors (language model, speech, embeddings)

This is where most real-world scale problems occur, because vendors enforce rate limits and have their own load. Standard practices:

Practice	What it protects against
Agreed quotas and capacity with each vendor	Being throttled at the exact moment traffic peaks. Reserved or enterprise capacity should be agreed *before* launch, sized to expected peak concurrency.
Timeouts on every call	One slow vendor call holding a conversation open. Short, stage-specific limits (e.g. a few seconds for embedding).
Retries with back-off and jitter	Brief vendor blips becoming user-visible failures. Retries are bounded and never applied blindly to anything unsafe to repeat.

Circuit breakers	A failing vendor dragging everything down. After repeated failures, calls stop for a short period and the fallback is used.
Fallback provider	A vendor outage. The adapter layer makes the swap a configuration change; the client's written consent and a latency re-benchmark apply.
Warm, persistent connections	Re-doing the secure-connection handshake on every call. Cuts a real, avoidable delay per turn.
Cache repeated embeddings	Paying the ~0.3–0.6 s embedding delay twice for identical text.
Same-region hosting	Cross-region round trips on every call. Host in Mumbai alongside India-based vendor endpoints.

> **HONEST NOTE**
> The service has per-call timeouts today (e.g. 3 s for embeddings, 30 s for the language model). It does **not yet** have retries, circuit breakers, or a connection that is kept warm for the language model (it currently opens a new connection per call). These are the first hardening items — see Section 10.

The embedding step deserves special mention

Each turn makes one embedding call to an external service. It is the biggest contributor to delay that we control, and it is a third-party dependency on the critical path. Options, in order: (1) keep the single shared embedding per turn (already done — it used to be repeated), (2) cache embeddings for repeated text, (3) agree a vendor quota, and (4) at much larger scale, host an embedding model ourselves in the same region. Option 4 would be a vendor substitution requiring the client's written sign-off and re-calibration of every safety threshold, so it is a deliberate later decision, not a casual optimisation.

5.5 Real-time voice (LiveKit)

- **Media servers:** LiveKit carries audio and does no thinking. It scales by adding media capacity. Managed cloud means a usage-based cost but little operational load; self-hosting gives control and needs a DevOps owner. Whichever is chosen, its region must be confirmed as compliant with data-residency requirements.
- **Voice-agent workers:** a pool of processes, each able to hold a number of simultaneous conversations. LiveKit dispatches each new room to a worker with free capacity; we autoscale the pool on active calls. Unlike a text request, a voice conversation holds resources for its entire duration, so capacity planning is by *concurrent calls*.
- **Pre-warming:** the voice-activity model is loaded once when a worker starts, not for each call, so new calls start quickly.
- **Interruptions (barge-in):** if the user speaks over the AI, the in-progress reply is cancelled cleanly so no wasted work continues in the background. Verified in tests.
- **Known improvement:** text-to-speech currently reconnects per sentence; keeping one warm connection per turn would reduce first-sound delay. It is a known, deferred item.
- **Recording:** only practitioner and emergency calls are recorded, not every AI conversation — recording everything would add large storage and cost with no product need.

5.6 The user-facing speed tricks (all standard practice)

Technique	Effect
Stream everything	Reply words are sent as they are produced; voice starts speaking from the first completed sentence. Perceived speed is time to *first* response, not total time — a reply that starts at 0.8 s and keeps flowing feels faster than one appearing whole at 1.8 s.
Filler phrases	On a slower turn, a short natural phrase (“Hmm, let me think about that...”) plays while the real reply is prepared. They are chosen by rule, never invented by the AI, and never mention technical terms. Currently a representative subset; the full 13-category library is still to be ported.
Parallel work	Independent steps (e.g. knowledge lookups) run together instead of one after another, and audit/summary work runs in the background.
Lean prompts	Send the language model recent turns plus a summary, not the whole transcript; send only the few passages that matter. Smaller prompts are faster and cheaper.
Safety by construction	The safety check is rule-based and local to our database, so it is fast *because of how it is built* — it is not sped up by making it less thorough.

6. Capacity planning: how we turn “many users” into numbers

“Many users” needs to become a concrete figure so we can size infrastructure and agree vendor quotas. The standard method uses a simple relationship (Little’s Law): **average conversations in progress = arrival rate × time each takes**.

Worked illustration (assumptions, not measurements)

> **READ AS AN EXAMPLE ONLY**
> These figures show *how* the calculation works. The real inputs must come from the client’s expected usage, and the real per-copy capacity from our load test.

Assumption	Value used in this example
Users active at the same moment (peak)	1,000
Each active user sends a message about every	20 seconds
Each reply occupies the service for about	2 seconds

Result: about 1,000 ÷ 20 = **50 new replies per second** arriving; each lasting ~2 s, so roughly **100 replies in progress at any instant**. Because replies are mostly waiting on vendors, a single service copy can usually hold many such replies at once. The load test tells us how many; we then run enough copies, plus spare headroom, to keep response time flat. The same arithmetic gives the **requests per second we must agree with each vendor**.

For voice, the unit is different: **1,000 simultaneous callers means 1,000 open voice sessions**, each holding a speech-to-text and a text-to-speech stream for the whole call. Voice capacity is planned by concurrent calls and is the more demanding case.

The inputs we need from the client

#	Question	Why it matters
1	Expected registered users at launch, in 6 months, in 12 months?	Sets growth path and when each phase is needed.
2	Expected peak *simultaneous* users, and when (evenings? exam season?)	Sizes copies and vendor quotas. Peak, not average, drives cost and risk.
3	Split between text and voice (and avatar, if enabled)?	Voice and avatar are far heavier per user than text.
4	Latency promise to users: is 1.5–2.5 s the commitment, and is P95 the measure?	Defines the service targets we alert on.
5	Availability target (e.g. 99.5% or 99.9% a month)?	Determines redundancy, and cost.
6	Budget ceiling per active user per month?	Drives cost levers and the choice of default mode.

7	Will usage be India only? Any plan outside India?	Data-residency rules limit multi-region options.
8	Any events that could create sudden spikes (campaigns, partner launches)?	Spikes need pre-warmed capacity, not just autoscaling.

7. Keeping it reliable: resilience and graceful degradation

At scale, something is always failing somewhere. The goal is that a failure becomes a small, controlled loss of a nice-to-have, never a loss of safety.

7.1 The degradation ladder

When capacity is stressed or a vendor is slow, we step down from the most expensive experience to the most reliable:

Level	Experience	When we step down
1 Avatar	Voice plus on-screen character (opt-in, cost-gated)	Avatar provider slow, cost gate, or heavy load.
2 Voice	Spoken conversation	Speech vendor slow or unavailable.
3 Text	Typed conversation	Always available as the final fallback.

Network quality is handled in the same spirit: video quality is reduced before audio, and audio before text, because responsiveness matters more than appearance.

7.2 Resilience practices

Practice	Purpose
Rate limiting per user	Stops one user or a faulty client from consuming shared capacity; also controls cost. Planned; not yet implemented.
Backpressure and queueing	Under extreme load, new requests wait briefly or get a polite “busy” response rather than every reply getting slower.
Timeouts, retries, circuit breakers	Contain vendor problems (Section 5.4).
Redundancy in every layer	At least two copies of the service; a standby database; replicated Redis; multiple voice workers.
Stateless recovery	If a service copy dies mid-conversation, the next message simply goes to another copy that reads the conversation state from Redis.
Safe deployments	Rolling updates, ability to roll back to the last approved knowledge-base version.

7.3 Isolation as a safety measure

The AI service is a separate deployable from the main product backend on purpose. A bug, deployment or outage in bookings, profiles or journaling cannot stop the safety check from running — and a surge of AI traffic cannot take down logins.

8. Safety under load — the non-negotiable rule

PRINCIPLE

Performance work must never weaken safety. The safety check runs on every message, including cache hits, at any load. We shed load from other places first.

Under heavy load we…	We never…
Step down avatar, then voice quality, toward text	Skip or shorten the safety check
Skip optional steps such as summaries and the extra “how long/affecting life” check	Reduce the red-flag trigger list to save time
Serve a repeat question from cache	Let a cache hit bypass the danger-phrase scan
Rate-limit or queue ordinary traffic	Delay a danger-phrase reply behind ordinary traffic
Use a generic safe reply if a vendor is down	Let the AI improvise a crisis response

What the code already guarantees

- The safety check is rule-based and runs against our own database. It does not depend on an external AI vendor being up, apart from the embedding step used to compare meanings.
- If the safety check itself fails or times out, the design is to **fail safe, upward** — default to the more cautious route — rather than skip it.
- A matched danger phrase sends the clinician’s exact script and never calls the language model, so the fastest path is also the safest.

DEPENDENCY TO BE EXPLICIT ABOUT

Danger-phrase matching compares meaning, so it relies on the embedding step. If the embedding vendor is unavailable, behaviour must be deliberately designed (e.g. a keyword-based backup check plus the cautious default). This is a specific resilience item on the hardening list, and one to review with the clinical team.

9. Measuring it: observability, targets and load testing

“You can’t scale what you can’t measure.” The service already records per-stage timings on every turn; the work is to expose and test against them.

9.1 What we measure on every turn

Metric	Status
Embedding, cache check, triage, tier scoring, safety check, language-model time and total (text mode)	**Recorded per turn**
Voice stages: speech-to-text first partial and final, text-to-speech first sound	**Not yet recorded**
P50 / P95 / P99 per stage on a dashboard	**To build**
Cache hit rate over time (a drop is an early warning that something upstream changed)	**To build**
Vendor error and throttle rates; queue depth; database pool usage; Redis memory	**To build**
Cost per conversation (tokens, speech minutes)	**To build**

9.2 Service-level objectives (proposed, for the client to approve)

Objective	Proposed target	Alert when
Text reply, first words (P95)	≤ 2.5 s	P95 above target for 5 minutes
Voice turn, speech end to first audio (P95)	≤ 2.5 s	Same
Danger-phrase reply (P99)	≤ 1 s	Any sustained breach — treated as urgent
Error rate	< 1% of turns	Above 1% for 5 minutes
Availability	To be agreed (e.g. 99.9%)	Error budget consumed too fast

Targets are a proposal based on the 1.5–2.5 s goal in the architecture documents. They become commitments only when the client agrees them.

9.3 Load testing — the evidence step

Before launch, and before each growth milestone, we run structured tests using a standard load-testing tool and simulated users:

1. **Baseline:** one service copy, increasing users until latency or errors rise — this gives conversations per copy.
2. **Target load:** the expected peak from Section 6, held for an hour, watching P95/P99.

3. **Stress:** push beyond peak to find the breaking point and confirm we degrade gracefully, not suddenly.
4. **Spike:** sudden jumps in users, to test autoscaling speed.
5. **Soak:** steady load for many hours to expose memory leaks and slow growth.
6. **Failure drills:** stop a service copy, slow a vendor, kill a database node — confirm recovery and that safety replies keep working.
7. **Voice:** simulated callers joining rooms, with real audio, measuring speech-to-reply time.

> **TEST WITH REAL CONTENT**
>
> Early live tests found that a short run of “successful” voice tests was actually returning silence. Load tests must check the *content* of replies (real speech, correct safety script), not just that bytes arrived.

10. Honest status: what is ready, and what is still to do

Area	Status	Detail
Stateless service design, shared memory in Redis	Done	Conversation state is in Redis, not in the process.
Streaming at every stage (text and voice)	Done	Text streams; sentence-by-sentence speech; verified live.
One embedding per turn, reused everywhere	Done	Cut repeated vendor calls.
Audit writes in the background	Done	Never blocks a reply.
Exact-match cache	Done	Safety still runs on a hit.
Interrupt (barge-in) cancels work cleanly	Done	Verified under task cancellation.
Per-stage latency recorded (text)	Done	Embedding, triage, tier scoring, safety, model, total.
Provider adapters and fallback hook	Partial	Adapters exist; automatic failover is not built.
Semantic cache	Designed	Placeholder; not yet implemented.
Database pool sizing and a pooler	To do	Currently library defaults; needs explicit sizing and a pooler.
Vector index for growth	To do	Exact scan today (fine at ~8,500 records); add and benchmark an index before growth.
Multiple worker processes per container, autoscaling	To do	Runs a single process today.
Warm connection to the language model	To do	Opens a new connection per call today.
Retries, circuit breakers, per-user rate limiting	To do	Only per-call timeouts exist.
Voice stage timing and one warm speech connection per turn	To do	Known, deferred.
Full filler-phrase library	To do	Representative subset only.
Backup safety check if embedding vendor is down	To design	Needs clinical input.
Dashboards, alerts, SLOs	To do	Data exists; dashboards do not.
Formal load test	Not done	No concurrent-user test has been run. All figures so far are single-conversation.
Automated regression tests for the live path	Not done	Live checks so far were manual scripts.

Production infrastructure (cloud environment, zones, managed services)	**Not set up**	Local containers only; production hosting is a separate workstream.

SAY THIS

“The design is scale-ready and the per-conversation speed is good. What we have not yet done is prove it with many simultaneous users — that is the first thing on the plan, and it is how we will replace our estimates with measured numbers.”

11. Phased roadmap

Phase	Work	Exit criteria (what “done” means)
Phase 0 — Baseline now	Instrument voice stages; build latency dashboards; write the load-test scripts; agree usage numbers and SLOs with the client (Section 6, 9.2).	Measured P50/P95/P99 for every stage; agreed targets.
Phase 1 — Launch hardening	Multiple worker processes and 2+ copies across two zones; explicit database pool sizes plus a pooler; warm connection to the language model; retries, circuit breakers and bounded timeouts; per-user rate limits; managed Redis and Postgres with standby; vendor quotas agreed; vector index added and verified against safety tests; embedding cache; backup safety check designed with clinicians; full filler library.	Load test passes at 2× expected launch peak with P95 within target; failure drills pass; safety replies verified under load.
Phase 2 — Growth	Autoscaling on live signals; read replicas for knowledge lookups; semantic cache; voice-worker autoscaling by active calls; cost dashboards; one warm speech connection per voice turn; regression suite in the delivery pipeline.	Sustained growth with flat latency; cost per active user within budget.
Phase 3 — Large scale	Partition and archive audit tables; evaluate a self-hosted embedding model (client sign-off and re-calibration required); evaluate a dedicated vector engine only if measured; capacity reviews each quarter.	Decisions driven by measurements, not assumptions.

Triggers: when we move from one phase to the next

Signal	Response
P95 latency above target for a sustained period	Check the per-stage breakdown; scale out; look at vendor throttling first.
Database connections or CPU above ~70% of capacity	Add the pooler / replicas; tune the heaviest queries.
Knowledge base grows by a large multiple	Add and benchmark a vector index; re-test safety matches.
Vendor throttle errors appear	Raise quotas; add fallback routing; improve caching.
Cost per active user rises	Review cost levers (below).

12. Cost at scale

Performance and cost are linked: most per-conversation cost is vendor usage, so the same techniques that make replies faster usually make them cheaper.

Cost driver	Scales with	Lever
Language model	Tokens per reply	Trim context to recent turns plus a summary; send only the few relevant passages; keep the standing instructions lean; cache generic replies.
Speech-to-text and text-to-speech	Minutes of audio	Compressed audio; voice is opt-in per user; default to text where appropriate.
Embeddings	Turns	One per turn (done); cache repeats.
Avatar	Rendered minutes	Opt-in and cost-gated by design; the most expensive component.
Voice media servers	Concurrent calls and minutes	Right-size the pool; only record practitioner and emergency calls.
Compute and database	Concurrent load	Autoscale in as well as out; reserve baseline capacity; partition and archive old audit data.
Abuse and runaway use	Few heavy users	Per-user rate limits and token metering.

Cost per active user per month should be on a dashboard from the first load test, so that any change that raises it is visible immediately.

13. Likely questions from the client, with answers

“How many users can it handle?”

We don’t quote a number we haven’t measured. The architecture lets capacity grow by adding copies, so the real limit is set by vendor quotas, voice capacity and budget — all of which we agree up front against the client’s expected peak. The load test will give a measured figure per service copy.

“Will it get slower as we add users?”

Not if capacity is added ahead of demand, which is what autoscaling, headroom and the load tests are for. We watch the slowest 5% of replies, not just the average, and alert before users notice.

“What is the weakest point?”

Third-party vendor capacity and their rate limits — the language model, speech services and the embedding service — rather than our own logic. That is why quotas, timeouts, retries, caching and a fallback provider are the first items on the hardening list.

“What happens if a vendor goes down?”

Calls time out quickly and a fallback provider is used if one is agreed. If voice is affected, users are stepped down to text. The safety path uses our own database, and a backup approach for the embedding step is on the plan for clinical review.

“Will voice scale as easily as text?”

It scales, but it is heavier: each call holds resources for its whole length, so voice is planned by concurrent calls and is the main driver of cost and infrastructure.

“Does a bigger knowledge base slow things down?”

Not at the scale involved. We are around 8,500 records, and the database approach we use stays fast into the tens of millions. We will add an index and re-test safety matches well before size becomes an issue.

“Is the safety check ever skipped to save time?”

Never. It runs on every message, including cached answers. Under load we reduce avatar, voice quality and optional steps first.

“Is the data still in India as you scale?”

Yes — the plan stays within the Mumbai region and uses managed services in that region. Anything that would move data elsewhere, or change a vendor, needs explicit review and the client’s written consent.

“What do you need from us?”

The usage figures in Section 6, agreement on the service targets in Section 9.2, support agreeing vendor quotas, and a decision on managed versus self-hosted voice infrastructure.

Glossary

Term	Meaning
Latency	Delay between a user finishing and the first part of the reply.
P50 / P95 / P99	Typical, slowest 1-in-20 and slowest 1-in-100 response times.
Horizontal scaling	Adding more identical copies instead of one bigger machine.
Stateless	No conversation memory kept inside a single server, so any copy can serve any user.
Load balancer	Spreads incoming requests across copies.
Autoscaling	Adding or removing copies automatically from live load.
Connection pooling	Sharing a small number of database connections among many service copies.
Read replica	A copy of the database used only for reading, to share the load.
Rate limit	A cap on requests per time — imposed by vendors on us, and by us on each user.
Circuit breaker	Automatically stops calling a failing service for a short time and uses a fallback.
Backpressure	Slowing or queueing new requests so the system is not overwhelmed.
Graceful degradation	Losing a nice-to-have feature instead of failing completely.
SLO	Service-level objective: an agreed measurable target, such as P95 ≤ 2.5 s.
Load test	Simulating many users to measure how the system behaves.
Embedding	A numeric fingerprint of a message’s meaning, used to match it to the knowledge base.

Sources: project documents on latency and performance, architecture, technology stack, data model, LiveKit real-time layer, the vector database comparison and the conversation-layer plan; plus the current backend code (configuration, orchestrator, cache, database session, provider adapters). Capacity figures marked “example” or “illustrative” are not measurements. Safety thresholds and scoring values referred to elsewhere remain provisional pending clinical review.